

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Fri 4 September 2015 - Part Theory

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

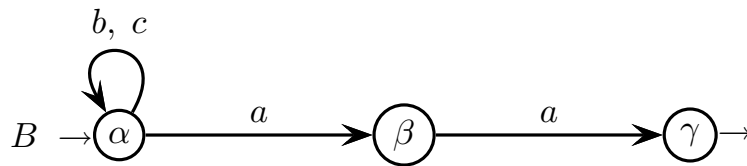
1 Regular Expressions and Finite Automata 20%

1. Given the alphabet $\Sigma = \{ a, b, c \}$, consider the following regular expression R :

$$R = c (b \mid b a)^* a$$

Answer the following questions:

- (a) Derive the deterministic automaton A that recognizes the language $L(R)$, by using the Berry-Sethi method.
- (b) Determine, in a systematic way, if the automaton A is minimal.
- (c) Consider the automaton B drawn below:



Design the automaton C that accepts the intersection of languages $L(R)$ and $L(B)$, by using the automaton cross product (Cartesian product) construction.

- (d) From the automaton C derived at the previous point, derive the regular expression R' for language $L(R) \cap L(B)$, by using the *BMC* method.
 - (e) (optional) Determine if the language $L(R)$ is local and provide an adequate justification to your answer.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the free language L , over the three-letter alphabet $\Sigma = \{ a, b, c \}$, defined as follows:

$$L = \{ x c w c y \mid x, y \in \{ a, b \}^* \wedge w^R = y x \}$$

Answer the following questions:

- (a) Write a non-extended (*BNF*) grammar G that generates the language L .
- (b) Consider the following language L' , which is the intersection of the language L with a regular language:

$$L' = L \cap (a b)^* c \Sigma^* c (a b)^*$$

Write a non-extended (*BNF*) grammar G' that generates the language L' .

- (c) (optional) Consider the language $L'' = \overline{L'}$, which is the complement of the language L' found at the previous point. Say if language L'' is regular or not, and reasonably justify your answer.
-

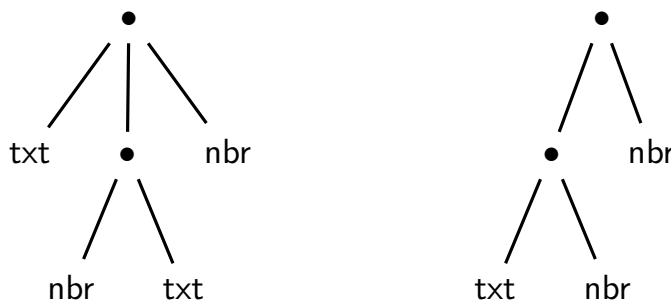
2. A marked language, inspired to *XML*, is used to encode trees in textual form. Such textual representations are subject to the following syntax constraints.

- All tree nodes are either ternary or binary inner nodes, or leaves. In particular:
 - ternary nodes have three child nodes, and are marked with the `<3TRSTRT>` and `<3TREND>` tags
 - binary nodes have two child nodes, and are marked with the `<2TRSTRT>` and `<2TREND>` tags
- Ternary nodes can have any type of child node, while the child nodes of binary nodes can only be binary nodes or leaves.
- Leaf nodes are either text elements (represented by the `txt` terminal) or number elements (represented by the `nbr` terminal).
- A phrase of the language consists of a non-empty, comma-separated, list of trees (that is, a so-called “tree forest”).

The sample text below encodes a list of two trees, the first with a ternary root and a binary inner node, and the second with a binary root and a binary inner node.

```
<3TRSTRT> txt <2TRSTRT> nbr txt <2TREND> nbr <3TREND> ,
<2TRSTRT> <2TRSTRT> txt nbr <2TREND> nbr <2TREND>
```

The figure below shows the two trees (forest) encoded by the text sample above.

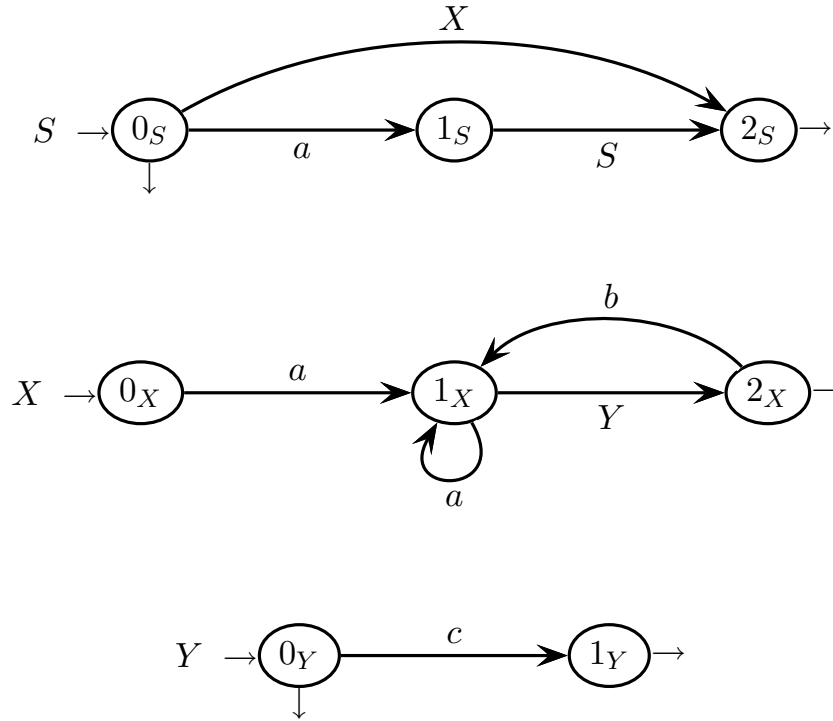


Answer the following questions:

- Write an extended grammar (*EBNF*) for this marked language and verify that the grammar is not ambiguous.
- (optional) Without changing the types and specifications of the ternary, binary and leaf nodes, define a new node type with an arity (number of child nodes) strictly greater than three. This new node type has tags `<>3TRSTART>` and `<>3TREND>`. Its child nodes can be of any type: leaf, binary, ternary, and the new node type. Show only the rules of the previous grammar that need to be modified, and the new rules introduced.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following *EBNF* grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ (axiom S):



Answer the following questions:

- (a) Draw the complete *ELR* pilot of grammar G , say if grammar G is of type *ELR* (1) or not, and justify your answer.
- (b) Find all the guide sets on all the arcs (terminal arcs, call arcs and exit arrows) of the machine net of grammar G , say if grammar G is of type *ELL* (1) or not, and justify your answer.
- (c) Say if grammar G is of type *ELL* (k) for some $k \geq 2$, and reasonably justify your answer.
- (d) (optional) Say if language $L(G)$ is of type *ELL* (k) for some $k \geq 2$, and reasonably justify your answer.

4 Language Translation and Semantic Analysis 20%

1. Given a binary string w , we denote as $\overline{w}$ the string with the bits complemented, and as w_2 the string where every bit is repeated. For instance, if $w = 0100$ then $\overline{w} = 1011$ and $w_2 = 00110000$.

Consider now the following translation τ :

$$\tau \left(w (\overline{w})^R \right) = \left((\overline{w})^R \right)_2$$

For instance, if $w = 0100$ then:

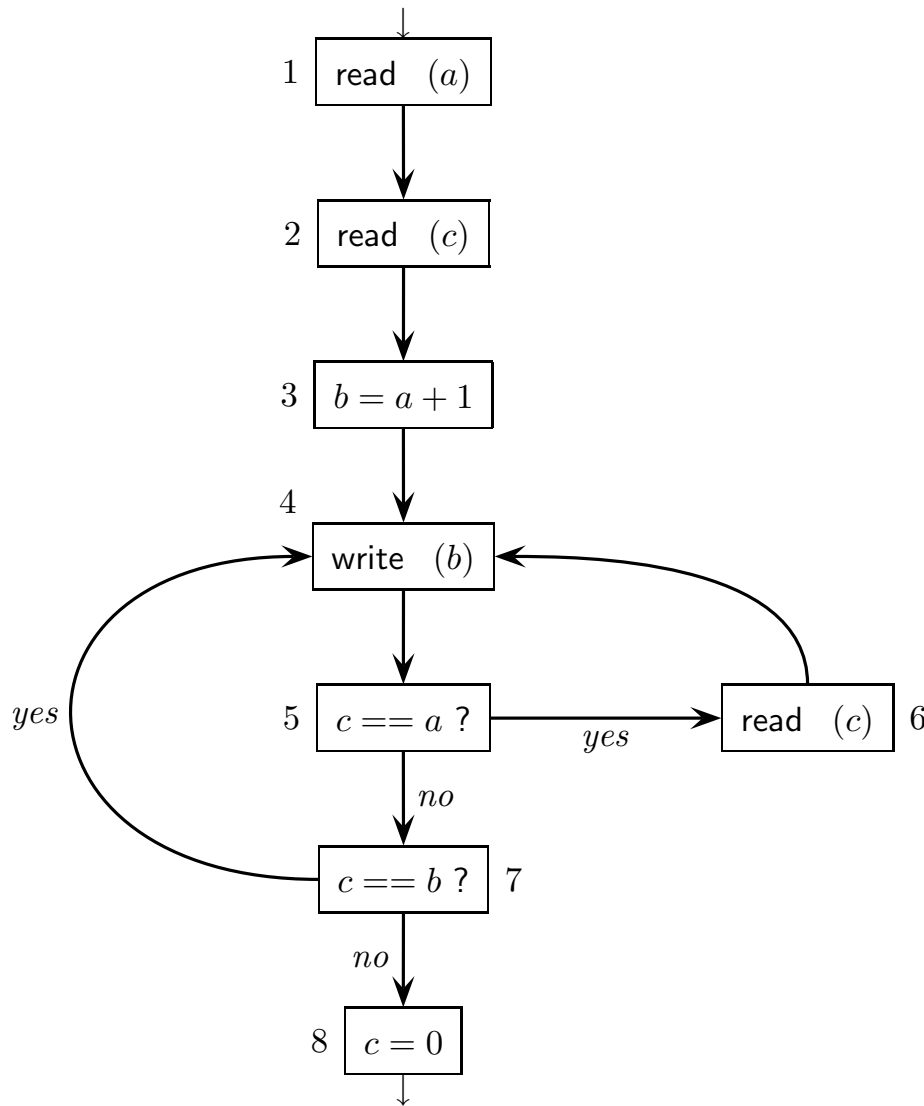
$$\tau \left(w (\overline{w})^R \right) = \tau(01001101) = (1101)_2 = 11110011$$

Notice that the strings of the source language are composed of a sequence of bits followed by the same sequence complemented and reversed.

Answer the following questions:

- (a) Design the syntactic translation scheme or grammar that defines translation τ .
 - (b) Draw the syntax tree (for the translation grammar) or trees (for the translation scheme) for the example $\tau(01001101) = (1101)_2 = 11110011$.
 - (c) (optional) Determine if the translation scheme or grammar is deterministic, and reasonably justify your answer.
-

2. Consider the following control flow graph of a program with three integer variables a , b and c :



Answer the following questions:

- Write the regular expression R , over the node alphabet $\{1, \dots, 8\}$, that models all the program execution traces, independently of the program semantics.
- Write the system of flow equations for the live variables of the program, solve such a system iteratively and write the variables live at each node (use the tables and graph prepared on the next pages).
- (optional) By considering the program semantics, write the regular expression R' of the effective execution traces modeled by the finite automaton represented by the graph, and say what changes to the distribution of the live variables are induced by the semantics.

please here write the **REGULAR EXPRESSION R**

tables of **DEFINED AND USED VARIABLES**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	
8		8	

system of data-flow equations for **LIVE VARIABLES**

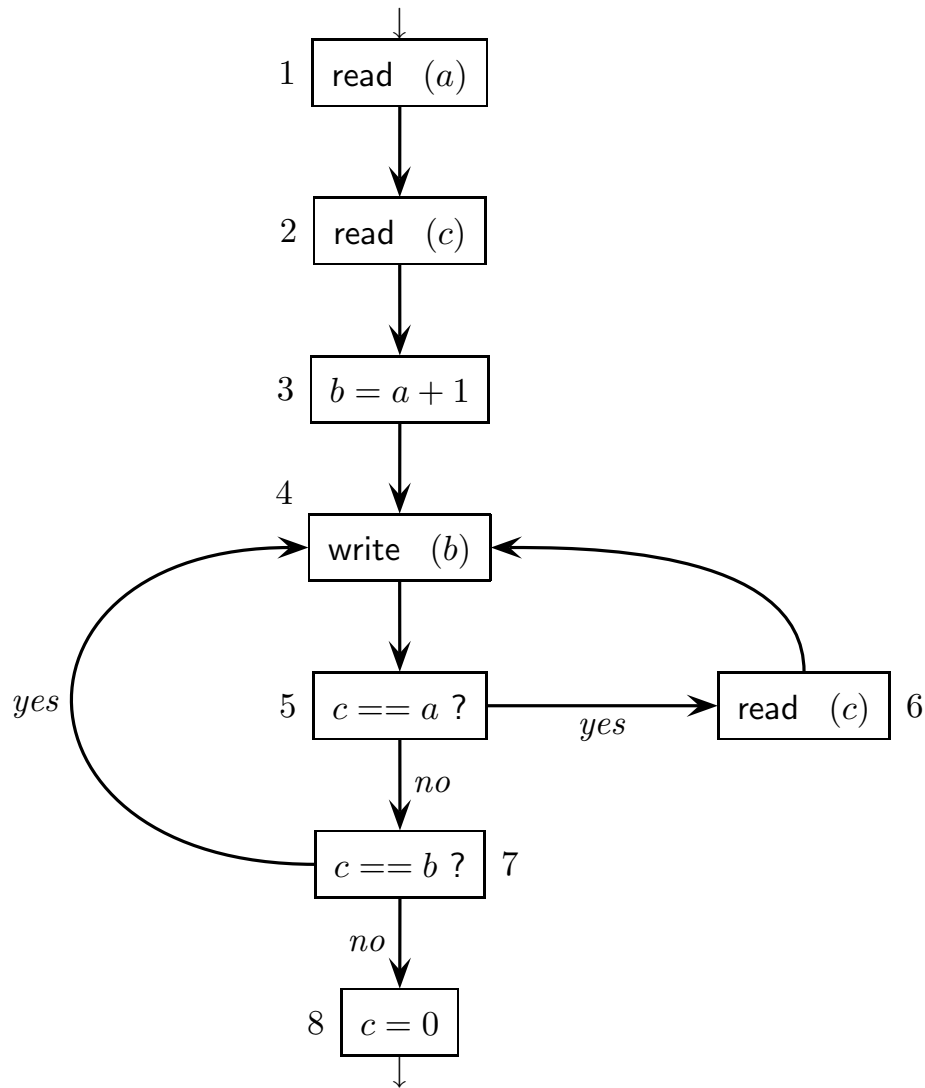
<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		
8		

iterative solution table of the system of data-flow equations (**LIVE VAR.S**)
(the number of tables and columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														
7														
8														

	7		8		9		10		11		12		13	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														
7														
8														

please here write the **LIVE VARIABLES**



please here write the **REGULAR EXPRESSION R'**

